Task 1:

edl_system			
Field Name	**Default Value**	**Units**	**Description**
altitude	np.NaN	m	Absolute altitude of the EDL system; updated dynamically during simulation.
velocity	np.NaN	m/s	Absolute velocity of the EDL system; updated dynamically during simulation.
num_rockets	8	-	The total number of retro-rockets on the EDL system.
volume	150	m^3	Total volume of atmospheric air displaced by the system, used for buoyancy calculations.
parachute	dict	-	Sub-dictionary defining the parachute subsystem.
heat_shield	dict	-	Sub-dictionary defining the heat shield subsystem.
rocket	dict	-	Sub-dictionary defining a single retro-rocket's properties.
speed_control	dict	-	Sub-dictionary defining the PID speed controller parameters.
position_control	dict	-	Sub-dictionary defining the PID altitude/position controller parameters.
sky_crane	dict	-	Sub-dictionary defining the hardware lowering the rover.

rover	dict	-	Sub-dictionary defining the rover payload (imported from define_rover_4).

parachute			
Field Name	**Default Value**	**Units**	**Description**
deployed	True	boolean	Indicates if the parachute is deployed (open) but not yet ejected.
ejected	False	boolean	Indicates if the parachute has been severed/released from the main system.
diameter	16.25	m	The nominal diameter of the parachute for drag area calculations.
Cd	0.615	-	The coefficient of drag for the parachute at subsonic speeds.
mass	185.0	kg	Total mass of the parachute hardware.

rocket			
Field Name	**Default Value**	**Units**	**Description**
on	False	boolean	Indicates whether the rocket thrusters are currently firing.
structure_mass	8.0	kg	The dry mass of a single rocket (excluding fuel).

initial_fuel_mass	230.0	kg	Starting mass of the propellant per rocket.
fuel_mass	230.0	kg	Current mass of the propellant per rocket; dynamically reduced during simulation.
effective_exhaust_velocity	4500.0	m/s	Efficiency metric determining the rate of mass loss for a given thrust.
max_thrust	3100.0	N	The upper saturation limit for thrust generation per rocket.
min_thrust	40.0	N	The lowest possible non-zero thrust output per rocket.

speed_control and position_control				
Dictionary	**Field Name**	**Default**	**Units**	**Description**
speed_control	on	False	boolean	Indicates if the PID speed targeting mode is active.
	Kp / Kd / Ki	2000 / 20 / 50	-	Proportional, Derivative, and Integral gain constants for speed correction.
	target_velocity	-3.0	m/s	The desired steady-state descent speed.

position_control	on	False	boolean	Indicates if the PID altitude targeting mode is active.
	Kp / Kd / Ki	2000 / 1000/ 50	-	Proportional, Derivative, and Integral gain constants for altitude correction.
	target_altitude	7.6	m	The desired hovering altitude, matched to the sky crane's max cable length.

sky_crane			
Field Name	**Default Value**	**Units**	**Description**
on	False	boolean	Indicates if the crane is actively lowering the rover.
danger_altitude	4.5	m	Minimum altitude at which rover touchdown is considered unsafe/damaging.
danger_speed	-1.0	m/s	Maximum descent speed at which touchdown would damage the rover.
mass	35.0	kg	Mass of the sky crane hardware.
area	16.0	m^2	Frontal area used for calculating drag once the heat shield is ejected.

Cd	0.9	-	Coefficient of drag for the sky crane body.
max_cable	7.6	m	The maximum extension length of the lowering cable.
velocity	-0.1	m/s	The constant relative velocity at which the crane lowers the rover.

heat_shield			
Field Name	**Default Value**	**Units**	**Description**
ejected	False	boolean	Indicates if the heat shield has been separated from the EDL structure.
mass	225.0	kg	Total mass of the heat shield.
diameter	4.5	m	Diameter of the shield, used to calculate drag area prior to ejection.
Cd	0.35	-	Coefficient of drag for the heat shield structure.

mars			
Field Name	**Default Value**	**Units**	**Description**
g	-3.72	m/s^2	Acceleration due to gravity on Mars.

altitude_threshold	7000	m	The altitude boundary used to switch between low and high atmosphere models.
low_altitude	dict	-	Atmospheric model definitions for altitudes < 7000 m.
high_altitude	dict	-	Atmospheric model definitions for altitudes > 7000 m.
density	function	kg/m^3	Lambda function calculating density from local temperature and pressure: p / (0.1921 * (T + 273.15)).

rover			
Field Name	**Default Value**	**Units**	**Description**
chassis	{'mass': 674}	kg	Dictionary containing the mass of the rover's main body structure.
science_payload	{'mass': 80}	kg	Dictionary containing the mass of the onboard scientific instruments.
power_subsys	{'mass': 100}	kg	Dictionary containing the mass of the power generation/storage system.
wheel_assembly	dict	-	Container for the drivetrain components.

wheel_assembly				
Component	**Field**	**Default**	**Units**	**Description**

wheel	radius	0.20	m	Physical radius of a single rover wheel.
	mass	2	kg	Mass of a single wheel.
speed_reducer	type	'reverted'	-	The architectural type of the gearing mechanism.
	diam_pinion	0.04	m	Diameter of the pinion gear.
	diam_gear	0.06	m	Diameter of the main gear.
	mass	1.5	kg	Mass of a single speed reducer.
motor	torque_stall	165	-	The stall torque of the motor.
	torque_noload	0	-	The torque at no-load operating conditions.
	speed_noload	3.85	-	The speed of the motor at no-load.
	mass	5.0	kg	Mass of a single motor.
	effcy_tau	[0, 10, 20...170]	-	A numpy array mapping efficiency torque curve data points.
	effcy	[0, .60, .75...05]	-	A numpy array of efficiency multipliers corresponding to the effcy_tau values.

mission_events			
Field Name	**Default Value**	**Units**	**Description**
alt_heatshield_eject	8000	m	The altitude trigger mapping to Event 0 (Heat shield ejection).
alt_parachute_eject	900	m	The altitude trigger mapping to Event 1 (Parachute separation).
alt_rockets_on	1800	m	The altitude trigger mapping to Event 2 (Initiation of retro-rocket firing).
alt_skycrane_on	7.6	m	The altitude trigger mapping to Event 3 (Sky crane begins lowering the rover).

Task 2:

Function Name	Calling Syntax	Description	Inputs	Outputs
get_mass_rover	m = get_mass_rover(edl_system)	Computes the total mass of the rover payload, including wheel assemblies, chassis, science payload, and power subsystem.	edl_system (dict): Main system parameters.	m (scalar float): Rover mass in kg.

get_mass_rockets	m = get_mass_rockets(edl_system)	Calculates the total mass of all retro-rockets, combining structural mass with dynamically updating fuel mass.	edl_system (dict): Main system parameters.	m (scalar float): Total rocket mass in kg.
get_mass_edl	m = get_mass_edl(edl_system)	Computes total active mass of the EDL system. Dynamically includes parachute and heat shield only if they have not been ejected.	edl_system (dict): Main system parameters.	m (scalar float): Total dynamic EDL mass in kg.
get_local_atm_properties	density, temperature, pressure = get_local_atm_properties(planet, altitude)	Returns local atmospheric properties using mathematical models based on altitude threshold. Not vectorized.	planet (dict): Planetary parameters. altitude (scalar float): Absolute altitude in meters.	density (kg/m^3) temperature (C) pressure (KPa).

Function Name	Calling Syntax	Description	Inputs	Outputs

F_buoyancy_descent	F = F_buoyancy_descent(edl_system, planet, altitude)	Computes net buoyancy force using displaced volume and local density	edl_system (dict) planet (dict) altitude (scalar float, m).	F (scalar float): Buoyancy force in N.
F_drag_descent	F = F_drag_descent(edl_system, planet, altitude, velocity)	Computes aerodynamic drag accounting for parachute deployment and heat shield ejection states.	edl_system (dict) planet (dict) altitude (float, m) velocity (float, m/s).	F (scalar float): Net drag force in N.
F_gravity_descent	F = F_gravity_descent(edl_system, planet)	Computes gravitational force using the dynamically updated total EDL mass.	edl_system (dict) planet (dict).	F (scalar float): Gravitational force in N.
v2M_Mars	M = v2M_Mars(v, a)	Converts descent speed to absolute Mach number using a PCHIP interpolated speed-of-sound vs. altitude table.	v (scalar float): Descent speed (m/s) a (scalar float): Altitude (m).	M (scalar float): Absolute Mach number.

Function Name	Calling Syntax	Description	Inputs	Outputs
thrust_controller	edl_system = thrust_controller(edl_system, planet)	Offline/reference PID control loop. Computes required thrust with gravity compensation and limits output based on motor saturation.	edl_system (dict) planet (dict).	edl_system (dict): Updated system dictionary with appended telemetry.
edl_events	events = edl_events(edl_system, mission_events)	Defines 9 lambda functions used by the ODE solver to trigger events.	edl_system (dict) mission_events (dict).	events (list): 9 terminal/directional lambda functions.
edl_dynamics	dydt = edl_dynamics(t, y, edl_system, planet)	Core ODE function calculating time derivatives of states, applying variable-mass physics and different control regimes.	t (float): Time (s) y (array): 7-element state vector edl_system (dict) planet (dict).	dydt (array): Time derivatives of the 7 state vector elements.
update_edl_state	edl_system, y0, TERMINATE_SIM = update_edl_state(...)	Processes ODE events, updates system hardware booleans, prints console logs, and resets initial conditions (y0).	edl_system (dict) TE, YE: Event arrays Y: Complete state history ITER_INFO (bool).	edl_system (dict) y0 (array): New initial conditions TERMINATE_SIM (bool).

simulate_edl	T, Y, edl_system = simulate_edl(...)	Top-level simulation loop sequentially calling solve_ivp and updating states via update_edl_state until termination conditions are met.	edl_system (dict) planet (dict) mission_events (dict) tmax (float, s) ITER_INFO (bool).	T (array): Time history Y (array): State history edl_system (dict): Final state.

<u>Task 3:</u>

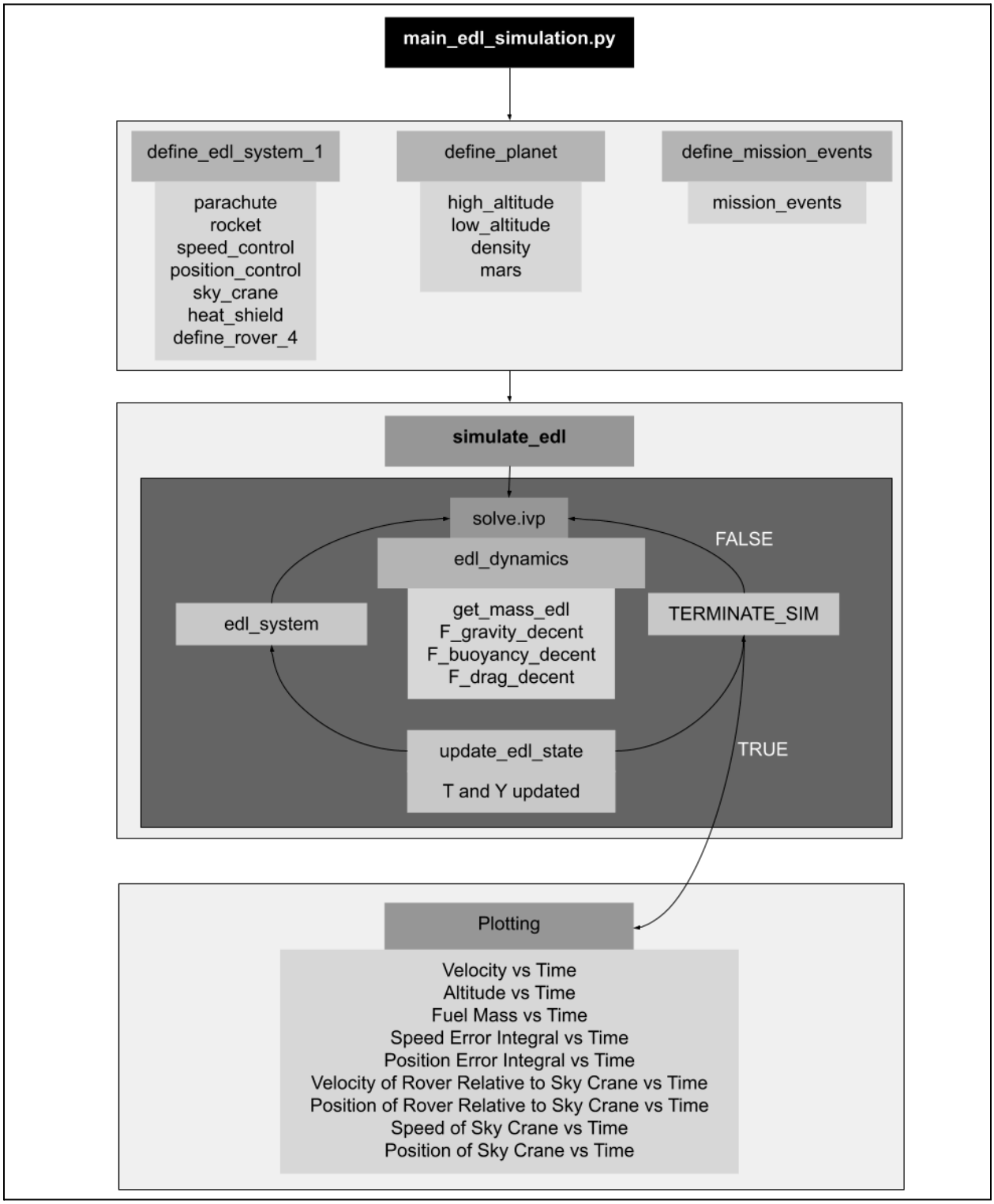
During each loop pass, the ODE (*solve_ivp*) solver, using the 'DOP853' solving method, solves the differential equations found in *edl_dynamics*. The 'DOP853' method uses an explicit 8th-order RK method, which offers higher precision compared to the standard 'RK45' method, which is only a 4th-order solver.

While running, the solver continually references the *edl_system* dictionary to know which physics to apply. For example, it can check *edl_system['position_control']['on']* to determine whether or not to use the PID position control. This is just one example of a check. There are many more for the status of sky crane or rocket. Depending on these states, the respective physics will be applied.

Additionally, during each iteration of the while loop, *Termination_SIM* is potentially updated if the appropriate break conditions are met.

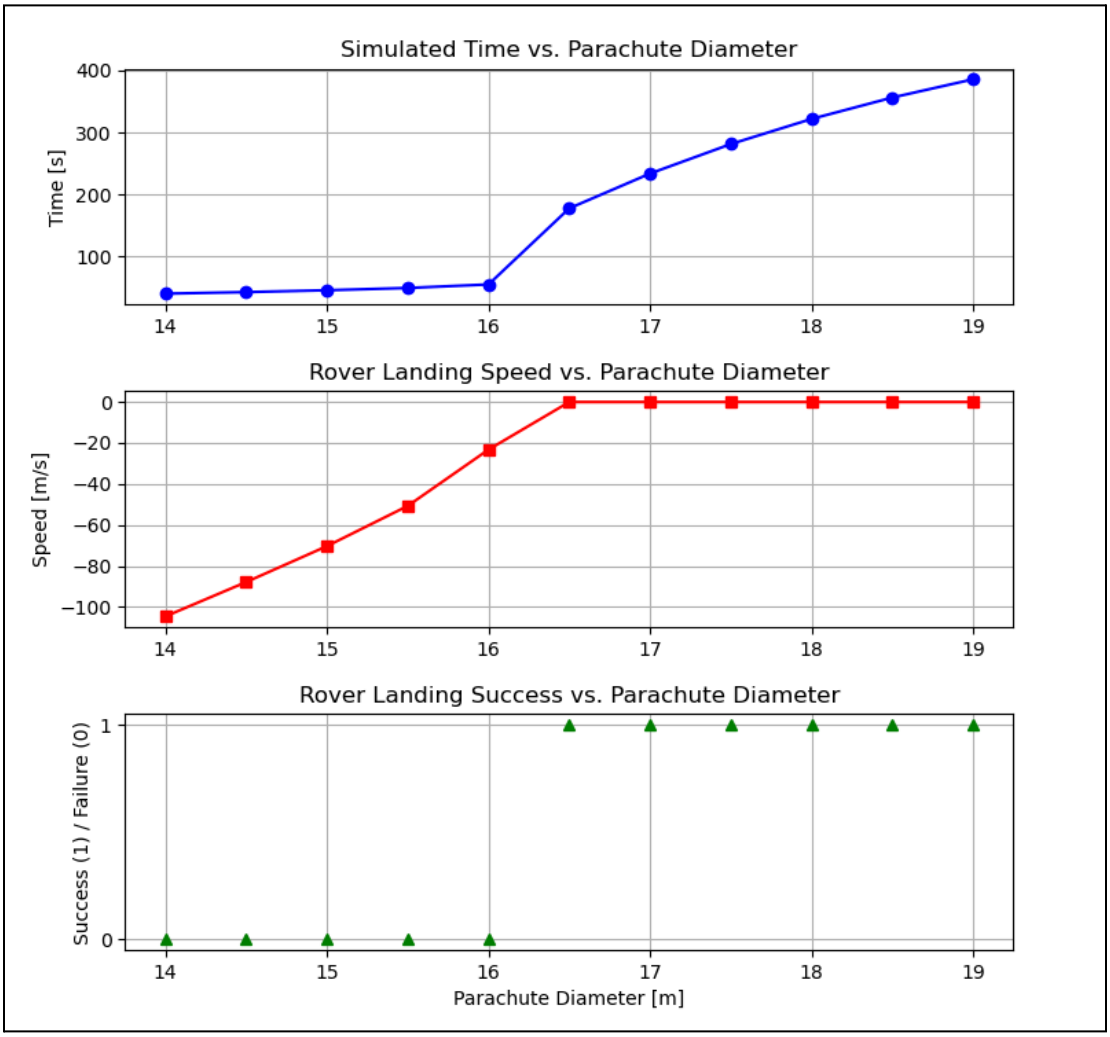
Because *solve_ivp* is stopped and restarted multiple times, the output arrays (*T* and *Y*) are not continuous. To prevent data loss, after each solution, the outputs are appended to a master list that maintains the data for all solutions.

Task 4:


main_edl_simulation.py
define_edl_system_1
parachute
rocket
speed_control
position_control
sky_crane
heat_shield
define_rover_4
define_planet
high_altitude
low_altitude
density
mars
define_mission_events
mission_events
simulate_edl
solve.ivp
edl_dynamics
get_mass_edl
F_gravity_decent
F_buoyancy_decent
F_drag_decent
edl_system
TERMINATE_SIM
FALSE
TRUE
update_edl_state
T and Y updated
Plotting
Velocity vs Time
Altitude vs Time
Fuel Mass vs Time
Speed Error Integral vs Time
Position Error Integral vs Time
Velocity of Rover Relative to Sky Crane vs Time
Position of Rover Relative to Sky Crane vs Time
Speed of Sky Crane vs Time
Position of Sky Crane vs Time

Task 5:

The team recommends a parachute diameter of 16.5 meters. This configuration ensures the rover reaches the ground safely while minimizing the time required to do so. The figures are shown below:



Task 6:

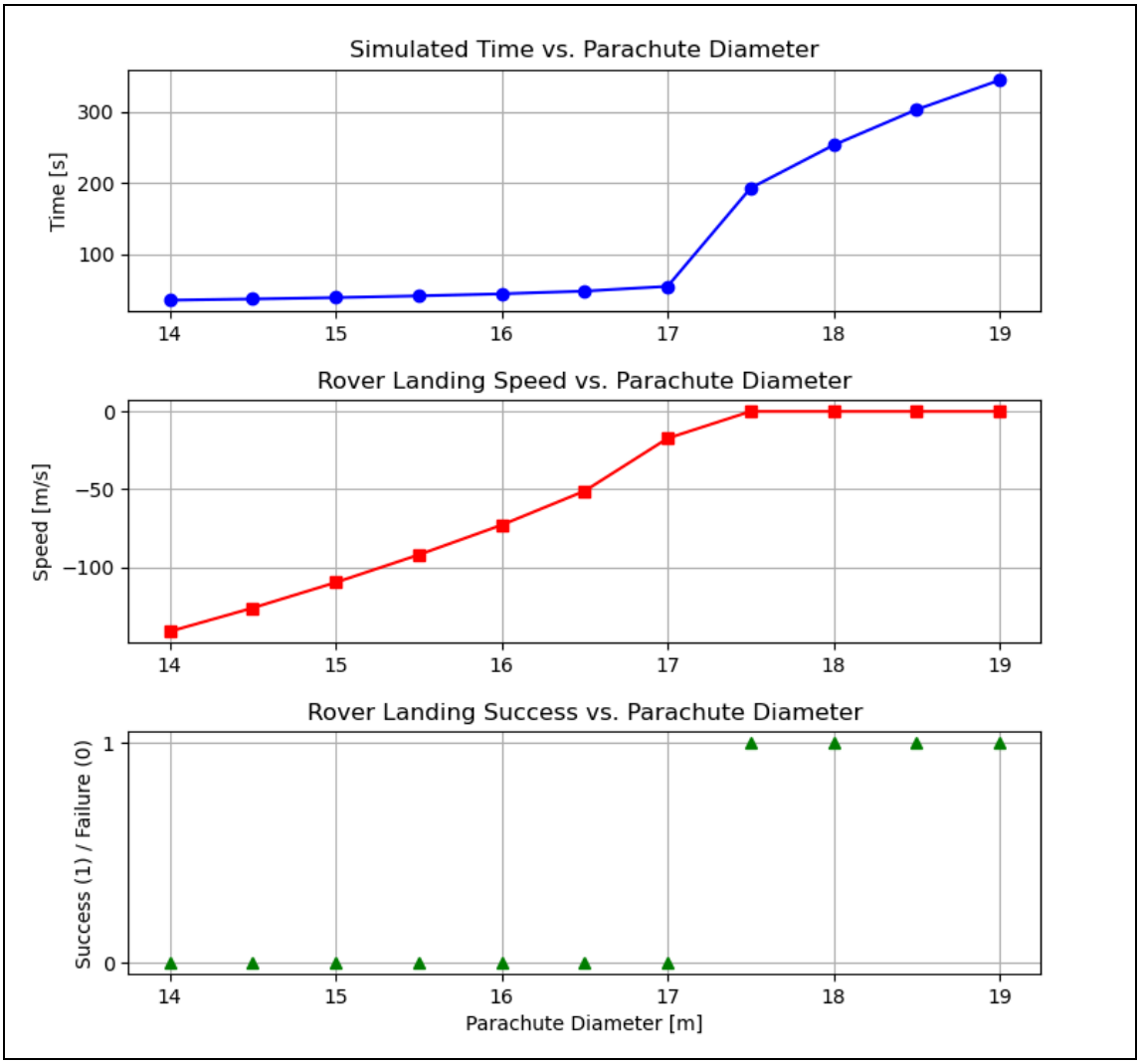
We created a continuous model for the MEF data using scipy.inderpl1d. This method was chosen as we have experience using this function from class labs, and it is the most efficient interpolation method, given that the dataset is not large. It would also prevent other assumptions from going outside the bounds set by the given data. The script from Task 5 remains intact, as use of the MEF data can be toggled on or off by changing the ‘use_mef’ key in the updated dictionary, as seen below (default is false):

```python
def define_edl_system_1():

    # parachute dict.  Includes physical definition and state information.
    # proper use is to set 'deployed' to False when parachute is ejected,
    # (as well as 'ejected' to True) in case someone checks deployed but not
    # ejected.
    # Note: we are starting the simulation from the point at which the
    # parachute is first deployed to keep things simpler.
    parachute = {'deployed' : True,  # true means it has been deployed but not ejected
                 'ejected' : False,  # true means parachute no longer is attached to system
                 'diameter' : 16.25, # [m] (MSL is about 16 m)
                 'Cd' : 0.615,       # [-] (0.615 is nominal for subsonic)
                 'mass' : 185.0,
                 'use_mef' : False}     # whether or not to use mef
```

Below is the actual process of implementing the MEF data

```python
# --- NEW MEF DRAG LOGIC ---
    # Checks if the 'use_mef' is True in the parachute configuration. If so, it applies a
    # Mach Efficiency Factor (MEF) to the parachute's ACd based on the current Mach number.
    # The MEF is determined by interpolating from a provided table of Mach numbers and
    # corresponding efficiency factors. This allows for a more accurate drag calculation
    # that accounts for changes in parachute performance at different speeds.
    if edl_system['parachute']['use_mef'] == True:
        # Data from the table in the project description. Mach numbers and corresponding MEF values.
        # Note that the Mach numbers should cover the range of expected velocities during descent,
        # and the MEF values should reflect the performance of the parachute at those speeds.
        mach_data = np.array([0.25, 0.5, 0.65, 0.7, 0.8, 0.9, 0.95, 1.0, 1.1,
                              1.2, 1.3, 1.4, 1.5, 1.6, 1.8, 1.9, 2.0, 2.2, 2.5, 2.6])

        mef_data = np.array([1.0, 1.0, 1.0, 0.97, 0.91, 0.72, 0.66, 0.75, 0.90,
                             0.96, 0.990, 0.999, 0.992, 0.98, 0.91, 0.85, 0.82, 0.75, 0.64, 0.62])

        # Convert current velocity to Mach number using the v2M_Mars function
        M = v2M_Mars(velocity, altitude)

        # Interpolate to find the current MEF based on Mach number;
        # have to set bounds_error=False and fill_value to handle cases where Mach number is outside the
        # provided data range. HAVE TO USE FILL_VALUE OTHEWISE IT BRICKS WHEN THE DIAMETER IS 19M FOR SOME REASON IDK WHYY
        mef_interp = interp1d(mach_data, mef_data, bounds_error=False, fill_value=(1.0, 0.62))
        MEF = mef_interp(M)

        # Apply the efficiency factor to the parachute's ACd
        ACd_parachute = ACd_parachute * MEF
```

With the MEF data being accounted for, the previously mentioned parachute diameter in Task 5 does require a change from an initial diameter of 16.5 meters to 17.5 meters.


Simulated Time vs. Parachute Diameter
Time [s]
300
200
100
14
15
16
17
18
19
Rover Landing Speed vs. Parachute Diameter
Speed [m/s]
0
−50
−100
14
15
16
17
18
19
Rover Landing Success vs. Parachute Diameter
Success (1) / Failure (0)
1
0
14
15
16
17
18
19
Parachute Diameter [m]